

choose $c = \lceil \ln |X| \rceil$. Since $i = ck$ is an upper bound on the number of iterations, which equals the size of $\mathcal{C}$, and $k = |\mathcal{C}^*|$, we have $|\mathcal{C}| \leq i = ck = c |\mathcal{C}^*| = |\mathcal{C}^*| \lceil \ln |X| \rceil$, and the theorem follows. ■

Exercises

35.3-1

Consider each of the following words as a set of letters: {arid, dash, drain, heard, lost, nose, shun, slate, snare, thread}. Show which set cover GREEDY-SET-COVER produces when you break ties in favor of the word that appears first in the dictionary.

35.3-2

Show that the decision version of the set-covering problem is NP-complete by reducing the vertex-cover problem to it.

35.3-3

Show how to implement GREEDY-SET-COVER to run in $O\left(\sum_{S \in \mathcal{F}} |S|\right)$ time.

35.3-4

The proof of Theorem 35.4 says that when GREEDY-SET-COVER, run on the instance $(X, \mathcal{F})$, returns the subfamily $\mathcal{C}$, then $|\mathcal{C}| \leq |\mathcal{C}^*| \lceil \ln |X| \rceil$. Show that the following weaker bound is trivially true:

$$|\mathcal{C}| \leq |\mathcal{C}^*| \max \{|S| : S \in \mathcal{F}\}.$$

35.3-5

GREEDY-SET-COVER can return a number of different solutions, depending on how it breaks ties in line 5. Give a procedure BAD-SET-COVER-INSTANCE(n) that returns an n -element instance of the set-covering problem for which, depending on how line 5 breaks ties, GREEDY-SET-COVER can return a number of different solutions that is exponential in n .

35.4 Randomization and linear programming

This section studies two useful techniques for designing approximation algorithms: randomization and linear programming. It starts with a simple randomized algorithm for an optimization version of 3-CNF satisfiability, and then it shows how to design an approximation algorithm for a weighted version of the vertex-cover problem based on linear programming. This section only scratches the surface of

these two powerful techniques. The chapter notes give references for further study of these areas.

A randomized approximation algorithm for MAX-3-CNF satisfiability

Just as some randomized algorithms compute exact solutions, some randomized algorithms compute approximate solutions. We say that a randomized algorithm for a problem has an *approximation ratio* of $\rho(n)$ if, for any input of size n , the *expected* cost C of the solution produced by the randomized algorithm is within a factor of $\rho(n)$ of the cost C^* of an optimal solution:

$$\max \left\{ \frac{C}{C^*}, \frac{C^*}{C} \right\} \leq \rho(n) \quad (35.11)$$

We call a randomized algorithm that achieves an approximation ratio of $\rho(n)$ a *randomized $\rho(n)$ -approximation algorithm*. In other words, a randomized approximation algorithm is like a deterministic approximation algorithm, except that the approximation ratio is for an expected cost.

A particular instance of 3-CNF satisfiability, as defined in Section 34.4, may or may not be satisfiable. In order to be satisfiable, there must exist an assignment of the variables so that every clause evaluates to 1. If an instance is not satisfiable, you might instead want to know how “close” to satisfiable it is, that is, find an assignment of the variables that satisfies as many clauses as possible. We call the resulting maximization problem *MAX-3-CNF satisfiability*. The input to MAX-3-CNF satisfiability is the same as for 3-CNF satisfiability, and the goal is to return an assignment of the variables that maximizes the number of clauses evaluating to 1. You might be surprised that randomly setting each variable to 1 with probability $1/2$ and to 0 with probability $1/2$ yields a randomized $8/7$ -approximation algorithm, but we’re about to see why. Recall that the definition of 3-CNF satisfiability from Section 34.4 requires each clause to consist of exactly three distinct literals. We now further assume that no clause contains both a variable and its negation. Exercise 35.4-1 asks you to remove this last assumption.

Theorem 35.5

Given an instance of MAX-3-CNF satisfiability with n variables $x_1, x_2, \dots, x_n$ and m clauses, the randomized algorithm that independently sets each variable to 1 with probability $1/2$ and to 0 with probability $1/2$ is a randomized $8/7$ -approximation algorithm.

Proof Suppose that each variable is independently set to 1 with probability $1/2$ and to 0 with probability $1/2$. Define, for $i = 1, 2, \dots, m$, the indicator random variable

$$Y_i = I\{\text{clause } i \text{ is satisfied}\} ,$$

so that $Y_i = 1$ as long as at least one of the literals in the i th clause is set to 1. Since no literal appears more than once in the same clause, and since we assume that no variable and its negation appear in the same clause, the settings of the three literals in each clause are independent. A clause is not satisfied only if all three of its literals are set to 0, and so $\Pr\{\text{clause } i \text{ is not satisfied}\} = (1/2)^3 = 1/8$. Thus, we have $\Pr\{\text{clause } i \text{ is satisfied}\} = 1 - 1/8 = 7/8$, and Lemma 5.1 on page 130 gives $E[Y_i] = 7/8$. Let Y be the number of satisfied clauses overall, so that $Y = Y_1 + Y_2 + \cdots + Y_m$. Then, we have

$$\begin{aligned} E[Y] &= E\left[\sum_{i=1}^m Y_i\right] \\ &= \sum_{i=1}^m E[Y_i] \quad (\text{by linearity of expectation}) \\ &= \sum_{i=1}^m 7/8 \\ &= 7m/8 . \end{aligned}$$

Since m is an upper bound on the number of satisfied clauses, the approximation ratio is at most $m/(7m/8) = 8/7$. ■

Approximating weighted vertex cover using linear programming

The *minimum-weight vertex-cover problem* takes as input an undirected graph $G = (V, E)$ in which each vertex $v \in V$ has an associated positive weight $w(v)$. The weight $w(V')$ of a vertex cover $V' \subseteq V$ is the sum of the weights of its vertices: $w(V') = \sum_{v \in V'} w(v)$. The goal is to find a vertex cover of minimum weight.

The approximation algorithm for unweighted vertex cover from Section 35.1 won't work here, because the solution it returns could be far from optimal for the weighted problem. Instead, we'll first compute a lower bound on the weight of the minimum-weight vertex cover, by using a linear program. Then we'll “round” this solution and use it to obtain a vertex cover.

Start by associating a variable $x(v)$ with each vertex $v \in V$, and require that $x(v)$ equals either 0 or 1 for each $v \in V$. The vertex cover includes v if and only if $x(v) = 1$. Then the constraint that for any edge (u, v) , at least one of u and v must belong to the vertex cover can be expressed as $x(u) + x(v) \geq 1$. This view gives rise to the following *0-1 integer program* for finding a minimum-weight vertex cover:

$$\text{minimize } \sum_{v \in V} w(v) x(v) \quad (35.12)$$

subject to

$$x(u) + x(v) \geq 1 \quad \text{for each } (u, v) \in E \quad (35.13)$$

$$x(v) \in \{0, 1\} \text{ for each } v \in V. \quad (35.14)$$

In the special case in which all the weights $w(v)$ equal 1, this formulation is the optimization version of the NP-hard vertex-cover problem. Let's remove the constraint that $x(v) \in \{0, 1\}$ and replace it by $0 \leq x(v) \leq 1$, resulting in the following linear program:

$$\text{minimize } \sum_{v \in V} w(v) x(v) \quad (35.15)$$

subject to

$$x(u) + x(v) \geq 1 \text{ for each } (u, v) \in E \quad (35.16)$$

$$x(v) \leq 1 \text{ for each } v \in V \quad (35.17)$$

$$x(v) \geq 0 \text{ for each } v \in V. \quad (35.18)$$

We refer to this linear program as the *linear-programming relaxation*. Any feasible solution to the 0-1 integer program in lines (35.12)–(35.14) is also a feasible solution to its linear-programming relaxation in lines (35.15)–(35.18). Therefore, the value of an optimal solution to the linear-programming relaxation provides a lower bound on the value of an optimal solution to the 0-1 integer program, and hence a lower bound on the optimal weight in the minimum-weight vertex-cover problem.

The procedure APPROX-MIN-WEIGHT-VC on the facing page starts with a solution to the linear-programming relaxation and uses it to construct an approximate solution to the minimum-weight vertex-cover problem. The procedure works as follows. Line 1 initializes the vertex cover to be empty. Line 2 formulates the linear-programming relaxation in lines (35.15)–(35.18) and then solves this linear program. An optimal solution gives each vertex v an associated value $\bar{x}(v)$, where $0 \leq \bar{x}(v) \leq 1$. The procedure uses this value to guide the choice of which vertices to add to the vertex cover C in lines 3–5: the vertex cover C includes vertex v if and only if $\bar{x}(v) \geq 1/2$. In effect, the procedure “rounds” each fractional variable in the solution to the linear-programming relaxation to either 0 or 1 in order to obtain a solution to the 0-1 integer program in lines (35.12)–(35.14). Finally, line 6 returns the vertex cover C .

Theorem 35.6

Algorithm APPROX-MIN-WEIGHT-VC is a polynomial-time 2-approximation algorithm for the minimum-weight vertex-cover problem.

APPROX-MIN-WEIGHT-VC(G, w)

```

1   $C = \emptyset$ 
2  compute  $\bar{x}$ , an optimal solution to the linear-programming relaxation
    in lines (35.15)–(35.18)
3  for each vertex  $v \in V$ 
4      if  $\bar{x}(v) \geq 1/2$ 
5           $C = C \cup \{v\}$ 
6  return  $C$ 

```

Proof Because there is a polynomial-time algorithm to solve the linear program in line 2, and because the **for** loop of lines 3–5 runs in polynomial time, APPROX-MIN-WEIGHT-VC is a polynomial-time algorithm.

It remains to show that APPROX-MIN-WEIGHT-VC is a 2-approximation algorithm. Let C^* be an optimal solution to the minimum-weight vertex-cover problem, and let z^* be the value of an optimal solution to the linear-programming relaxation in lines (35.15)–(35.18). Since an optimal vertex cover is a feasible solution to the linear-programming relaxation, z^* must be a lower bound on $w(C^*)$, that is,

$$z^* \leq w(C^*) . \quad (35.19)$$

Next, we claim that rounding the fractional values of the variables $\bar{x}(v)$ in lines 3–5 produces a set C that is a vertex cover and satisfies $w(C) \leq 2z^*$. To see that C is a vertex cover, consider any edge $(u, v) \in E$. By constraint (35.16), we know that $x(u) + x(v) \geq 1$, which implies that at least one of $\bar{x}(u)$ and $\bar{x}(v)$ is at least $1/2$. Therefore, at least one of u and v is included in the vertex cover, and so every edge is covered.

Now we consider the weight of the cover. We have

$$\begin{aligned}
 z^* &= \sum_{v \in V} w(v) \bar{x}(v) \\
 &\geq \sum_{v \in V: \bar{x}(v) \geq 1/2} w(v) \bar{x}(v) \\
 &\geq \sum_{v \in V: \bar{x}(v) \geq 1/2} w(v) \cdot \frac{1}{2} \\
 &= \sum_{v \in C} w(v) \cdot \frac{1}{2} \\
 &= \frac{1}{2} \sum_{v \in C} w(v) \\
 &= \frac{1}{2} w(C) .
 \end{aligned} \quad (35.20)$$

Combining inequalities (35.19) and (35.20) gives

$$w(C) \leq 2z^* \leq 2w(C^*),$$

and hence APPROX-MIN-WEIGHT-VC is a 2-approximation algorithm. ■

Exercises

35.4-1

Show that even if a clause is allowed to contain both a variable and its negation, randomly setting each variable to 1 with probability $1/2$ and to 0 with probability $1/2$ still yields a randomized $8/7$ -approximation algorithm.

35.4-2

The **MAX-CNF satisfiability problem** is like the MAX-3-CNF satisfiability problem, except that it does not restrict each clause to have exactly three literals. Give a randomized 2-approximation algorithm for the MAX-CNF satisfiability problem.

35.4-3

In the MAX-CUT problem, the input is an unweighted undirected graph $G = (V, E)$. We define a cut $(S, V - S)$ as in Chapter 21 and the **weight** of a cut as the number of edges crossing the cut. The goal is to find a cut of maximum weight. Suppose that each vertex v is randomly and independently placed into S with probability $1/2$ and into $V - S$ with probability $1/2$. Show that this algorithm is a randomized 2-approximation algorithm.

35.4-4

Show that the constraints in line (35.17) are redundant in the sense that removing them from the linear-programming relaxation in lines (35.15)–(35.18) yields a linear program for which any optimal solution x must satisfy $x(v) \leq 1$ for each $v \in V$.

35.5 The subset-sum problem

Recall from Section 34.5.5 that an instance of the subset-sum problem is given by a pair (S, t) , where S is a set $\{x_1, x_2, \dots, x_n\}$ of positive integers and t is a positive integer. This decision problem asks whether there exists a subset of S that adds up exactly to the target value t . As we saw in Section 34.5.5, this problem is NP-complete.

The optimization problem associated with this decision problem arises in practical applications. The optimization problem seeks a subset of $\{x_1, x_2, \dots, x_n\}$

whose sum is as large as possible but not larger than t . For example, consider a truck that can carry no more than t pounds, which is to be loaded with up to n different boxes, the i th of which weighs x_i pounds. How heavy a load can the truck take without exceeding the t -pound weight limit?

We start this section with an exponential-time algorithm to compute the optimal value for this optimization problem. Then we show how to modify the algorithm so that it becomes a fully polynomial-time approximation scheme. (Recall that a fully polynomial-time approximation scheme has a running time that is polynomial in $1/\epsilon$ as well as in the size of the input.)

An exponential-time exact algorithm

Suppose that you compute, for each subset S' of S , the sum of the elements in S' , and then you select, among the subsets whose sum does not exceed t , the one whose sum is closest to t . This algorithm returns the optimal solution, but it might take exponential time. To implement this algorithm, you can use an iterative procedure that, in iteration i , computes the sums of all subsets of $\{x_1, x_2, \dots, x_i\}$, using as a starting point the sums of all subsets of $\{x_1, x_2, \dots, x_{i-1}\}$. In doing so, you would realize that once a particular subset S' has a sum exceeding t , there is no reason to maintain it, since no superset of S' can be an optimal solution. Let's see how to implement this strategy.

The procedure EXACT-SUBSET-SUM takes an input set $S = \{x_1, x_2, \dots, x_n\}$, the size $n = |S|$, and a target value t . This procedure iteratively computes L_i , the list of sums of all subsets of $\{x_1, \dots, x_i\}$ that do not exceed t , and then it returns the maximum value in L_n .

If L is a list of positive integers and x is another positive integer, then let $L + x$ denote the list of integers derived from L by increasing each element of L by x . For example, if $L = \langle 1, 2, 3, 5, 9 \rangle$, then $L + 2 = \langle 3, 4, 5, 7, 11 \rangle$. This notation extends to sets, so that

$$S + x = \{s + x : s \in S\}.$$

EXACT-SUBSET-SUM(S, n, t)

```

1   $L_0 = \langle 0 \rangle$ 
2  for  $i = 1$  to  $n$ 
3       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
4      remove from  $L_i$  every element that is greater than  $t$ 
5  return the largest element in  $L_n$ 
```

EXACT-SUBSET-SUM invokes an auxiliary procedure MERGE-LISTS(L, L'), which returns the sorted list that is the merge of its two sorted input lists L and L' ,

with duplicate values removed. Like the MERGE procedure we used in merge sort on page 36, MERGE-LISTS runs in $O(|L| + |L'|)$ time. We omit the pseudocode for MERGE-LISTS.

To see how EXACT-SUBSET-SUM works, let P_i denote the set of values obtained by selecting each (possibly empty) subset of $\{x_1, x_2, \dots, x_i\}$ and summing its members. For example, if $S = \{1, 4, 5\}$, then

$$\begin{aligned} P_1 &= \{0, 1\} , \\ P_2 &= \{0, 1, 4, 5\} , \\ P_3 &= \{0, 1, 4, 5, 6, 9, 10\} . \end{aligned}$$

Given the identity

$$P_i = P_{i-1} \cup (P_{i-1} + x_i) , \quad (35.21)$$

you can prove by induction on i (see Exercise 35.5-1) that the list L_i is a sorted list containing every element of P_i whose value is not more than t . Since the length of L_i can be as much as 2^i , EXACT-SUBSET-SUM is an exponential-time algorithm in general, although it is a polynomial-time algorithm in the special cases in which t is polynomial in $|S|$ or all the numbers in S are bounded by a polynomial in $|S|$.

A fully polynomial-time approximation scheme

The key to devising a fully polynomial-time approximation scheme for the subset-sum problem is to “trim” each list L_i after it is created. Here’s the idea behind trimming: if two values in L are close to each other, then since the goal is just an approximate solution, there is no need to maintain both of them explicitly. More precisely, use a trimming parameter δ such that $0 < \delta < 1$. When **trimming** a list L by δ , remove as many elements from L as possible, in such a way that if L' is the result of trimming L , then for every element y that was removed from L , some element z still in L' approximates y . For z to approximate y , it must be no greater than y and also within a factor of $1 + \delta$ of y , so that

$$\frac{y}{1 + \delta} \leq z \leq y . \quad (35.22)$$

You can think of such a z as “representing” y in the new list L' . Each removed element y is represented by a remaining element z satisfying inequality (35.22). For example, suppose that $\delta = 0.1$ and

$$L = \langle 10, 11, 12, 15, 20, 21, 22, 23, 24, 29 \rangle .$$

Then trimming L results in

$$L' = \langle 10, 12, 15, 20, 23, 29 \rangle ,$$

where the deleted value 11 is represented by 10, the deleted values 21 and 22 are represented by 20, and the deleted value 24 is represented by 23. Because every element of the trimmed version of the list is also an element of the original version of the list, trimming can dramatically decrease the number of elements kept while keeping a close (and slightly smaller) representative value in the list for each deleted element.

The procedure TRIM trims list $L = \langle y_1, y_2, \dots, y_m \rangle$ in $\Theta(m)$ time, given L and the trimming parameter δ . It assumes that L is sorted into monotonically increasing order. The output of the procedure is a trimmed, sorted list. The procedure scans the elements of L in monotonically increasing order. A number is appended onto the returned list L' only if it is the first element of L or if it cannot be represented by the most recent number placed into L' .

```

TRIM( $L, \delta$ )
1  let  $m$  be the length of  $L$ 
2   $L' = \langle y_1 \rangle$ 
3   $last = y_1$ 
4  for  $i = 2$  to  $m$ 
5      if  $y_i > last \cdot (1 + \delta)$            //  $y_i \geq last$  because  $L$  is sorted
6          append  $y_i$  onto the end of  $L'$ 
7           $last = y_i$ 
8  return  $L'$ 

```

Given the procedure TRIM, the procedure APPROX-SUBSET-SUM on the following page implements the approximation scheme. This procedure takes as input a set $S = \{x_1, x_2, \dots, x_n\}$ of n integers (in arbitrary order), the size $n = |S|$, the target integer t , and an approximation parameter ϵ , where

$$0 < \epsilon < 1. \quad (35.23)$$

It returns a value z^* whose value is within a factor of $1 + \epsilon$ of the optimal solution.

The APPROX-SUBSET-SUM procedure works as follows. Line 1 initializes the list L_0 to be the list containing just the element 0. The **for** loop in lines 2–5 computes L_i as a sorted list containing a suitably trimmed version of the set P_i , with all elements larger than t removed. Since the procedure creates L_i from L_{i-1} , it must ensure that the repeated trimming doesn't introduce too much compounded inaccuracy. That's why instead of the trimming parameter being ϵ in the call to TRIM, it has the smaller value $\epsilon/2n$. We'll soon see that APPROX-SUBSET-SUM returns a correct approximation if one exists.

```

APPROX-SUBSET-SUM( $S, n, t, \epsilon$ )
1   $L_0 = \langle 0 \rangle$ 
2  for  $i = 1$  to  $n$ 
3       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
4       $L_i = \text{TRIM}(L_i, \epsilon/2n)$ 
5      remove from  $L_i$  every element that is greater than  $t$ 
6  let  $z^*$  be the largest value in  $L_n$ 
7  return  $z^*$ 

```

As an example, suppose that APPROX-SUBSET-SUM is given

$S = \langle 104, 102, 201, 101 \rangle$

with $t = 308$ and $\epsilon = 0.40$. The trimming parameter δ is $\epsilon/2n = 0.40/8 = 0.05$. The procedure computes the following values on the indicated lines:

line 1: $L_0 = \langle 0 \rangle$,
line 3: $L_1 = \langle 0, 104 \rangle$,
line 4: $L_1 = \langle 0, 104 \rangle$,
line 5: $L_1 = \langle 0, 104 \rangle$,
line 3: $L_2 = \langle 0, 102, 104, 206 \rangle$,
line 4: $L_2 = \langle 0, 102, 206 \rangle$,
line 5: $L_2 = \langle 0, 102, 206 \rangle$,
line 3: $L_3 = \langle 0, 102, 201, 206, 303, 407 \rangle$,
line 4: $L_3 = \langle 0, 102, 201, 303, 407 \rangle$,
line 5: $L_3 = \langle 0, 102, 201, 303 \rangle$,
line 3: $L_4 = \langle 0, 101, 102, 201, 203, 302, 303, 404 \rangle$,
line 4: $L_4 = \langle 0, 101, 201, 302, 404 \rangle$,
line 5: $L_4 = \langle 0, 101, 201, 302 \rangle$.

The procedure returns $z^* = 302$ as its answer, which is well within $\epsilon = 40\%$ of the optimal answer $307 = 104 + 102 + 101$. In fact, it is within 2%.

Theorem 35.7

APPROX-SUBSET-SUM is a fully polynomial-time approximation scheme for the subset-sum problem.